High Level Design (HLD) Document

Web Vulnerability Scanner Application

Revision Number: 1.0
Last Date of Revision: 26/06/2026

Submitted By:
Amal S

Document Version Control

Date Issued	Version	Description	Author
26/06/2026	1.0	Initial HLD for Capstone Project	

Contents

Abstract

Modern web applications face frequent and sophisticated security threats. Manually searching for common vulnerabilities like Cross-Site Scripting (XSS) and SQL Injection (SQLi) is tedious, prone to error, and requires specialized security expertise. The Web Vulnerability Scanner addresses this challenge by automating the discovery, testing, and documentation of entry points within a web application. The scanner crawls a web application, identifies inputs and forms, injects testing payloads, and analyzes responses to identify vulnerabilities. It compiles all findings into a clean text report, helping developers secure their applications during the development lifecycle.

1. Introduction

1.1 Why this is a High-Level Document

The purpose of this High-Level Design (HLD) Document is to provide an overall architectural view of the Web Vulnerability Scanner Application and define the major modules involved. This document serves as a reference during development, testing, and validation of the scanning tool, describing the data flow and module interactions.

The HLD will:

- Present the overall system architecture and workflow.
- Describe the scanning pipeline used for vulnerability detection.
- Explain the role of the crawler, form parser, and injection modules.
- Outline the error handling and reporting structure.
- List key non-functional requirements such as performance and safety constraints.

1.2 Scope

This HLD document covers the entire codebase of the Web Vulnerability Scanner, including:

- The target crawler logic for link discovery.
- The HTML form parsing engine.
- The signature-based XSS and SQLi detection modules.
- The report generation system.
- The mock target web application used for verification.

1.3 Definitions

Term	Description
XSS	Cross-Site Scripting; a vulnerability where malicious scripts are injected into benign websites.

Term	Description
SQLi	SQL Injection; a vulnerability where an attacker manipulates database queries via input fields.
BeautifulSoup	A Python library used for parsing HTML documents and extracting data.
Flask	A lightweight WSGI web application framework in Python, used here to build the test application.
Payload	The specific input data or script used to exploit or detect a vulnerability.
DOM	Document Object Model; the structural representation of an HTML document.

2. General Description

2.1 Product Perspective

The Web Vulnerability Scanner is a modular, lightweight Python-based security tool. It operates as a black-box testing utility that interacts with a target web application over HTTP. By crawling pages, extracting forms, and testing inputs, it mimics the initial phase of a penetration test, reporting security weaknesses without requiring direct access to the application's source code or database.

2.2 Problem Statement

Web application security is critical, yet manual pentesting is time-consuming and expensive. Small development teams require automated, low-overhead tools to quickly verify the security posture of their applications against common OWASP Top 10 vulnerabilities (such as Injection and Cross-Site Scripting) during the continuous integration and deployment phases.

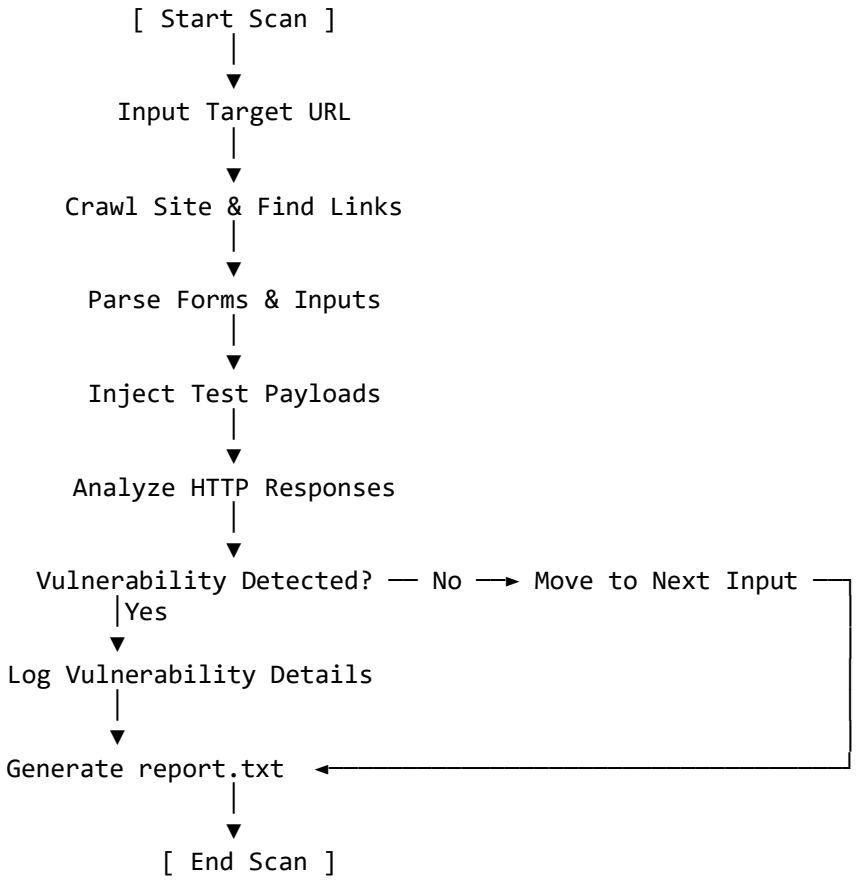
2.3 Proposed Solution

We implement an automated vulnerability scanner that integrates crawling, form parsing, payload injection, and response analysis into a single, cohesive workflow:

1. **Crawl:** The tool discovers all accessible pages within the target domain to build a map of the site.
2. **Parse:** It analyzes the HTML of each page to locate forms, query parameters, and input fields.
3. **Inject:** It submits specific security payloads (XSS scripts and SQLi control characters) to the identified input fields.
4. **Analyze:** It evaluates the HTTP responses to determine if the payload was executed (reflected) or if the database returned syntax errors.

5. **Report:** It logs all identified issues to a consolidated text report for developers.

The diagram below illustrates the end-to-end scanning workflow:



2.4 Further Improvements

The system can be enhanced in future iterations by:

- Adding support for more vulnerability classes, such as Local File Inclusion (LFI), Cross-Site Request Forgery (CSRF), and Broken Authentication.
- Implementing multi-threaded crawling and scanning to improve execution speed on larger target sites.
- Building a web-based or graphical user interface (GUI) to replace the command-line interface.
- Integrating cookie and session management to allow scanning of pages behind authentication walls.

2.5 Data Requirements

- **Input Data:** A target base URL (e.g., http://127.0.0.1:5000/) provided by the user.

- **Internal Data Structures:** Lists of discovered internal URLs, form metadata dictionaries (containing action, method, and input names), and arrays of test payloads.
- **Output Data:** A structured log printed to the terminal console and a persistent text report (report.txt) outlining the details of every vulnerability found.

2.6 Tools Used

1. **Python:** The core programming language used to build the scanner logic, processing pipelines, and mock web application.
2. **Requests:** An HTTP library for Python, used to send GET and POST requests, manage query parameters, and retrieve server responses.
3. **BeautifulSoup (BS4):** Used to parse HTML documents, crawl anchor tags (<a>), and extract form attributes (<form>, <input>).
4. **Flask:** A micro web framework, utilized to build the mock vulnerable web application for safe, local testing.

2.7 Constraints

- **Domain Constraint:** To prevent unauthorized or illegal scanning, the crawler must be restricted to the host domain of the initial target URL.
- **Signature Reliance:** The scanner relies on signature-based detection (specific reflected strings or database error patterns). It may not detect vulnerabilities that do not produce these specific signatures (e.g., blind SQL injection).
- **Single-Threaded Scanning:** The basic version scans sequentially. While safe, it may experience latency when scanning large applications with numerous forms.

3. Design Details

3.1 Process Flow

3.1.1 Target URL Input

The user executes the scanner and provides a target base URL. This URL serves as the seed for the crawler and defines the boundary of the security scan.

3.1.2 Web Crawling

The crawler module (crawler.py) downloads the seed page, parses it using BeautifulSoup, and extracts all anchor tags (<a>).

- **Relative to Absolute:** Relative links (e.g., /search) are resolved to absolute URLs (e.g., http://127.0.0.1:5000/search) using urllib.parse.urljoin.
- **Scope Filtering:** The crawler discards external links (e.g., https://github.com) to remain strictly within the target scope.

3.1.3 HTML Form Parsing

For each discovered page, the form parser (form_parser.py) extracts all <form> elements. It builds a structured dictionary for each form, capturing:

- **Action:** The target URL where the form submits data.
- **Method:** The HTTP method used (GET or POST).
- **Inputs:** The names and types of all interactive input fields (e.g., text, search, password).

3.1.4 Vulnerability Payload Injection

The scanner injects specific test payloads into the identified inputs:

- **XSS Injection:** Injects

```
<script>alert('XSS')</script>
```

- **SQLi Injection:** Injects

```
' OR '1'='1
```

3.1.5 Response Analysis

The scanner submits the forms with the injected payloads and analyzes the responses:

- **XSS Detection:** The scanner checks if the exact, unescaped payload string is present in the response body. If the browser would execute the script, the vulnerability is flagged.
- **SQLi Detection:** The scanner searches the response text for common database error signatures (such as "sql", "syntax", or "database"). If an error is triggered, the vulnerability is flagged.

3.1.6 Report Generation

The report generator (report_generator.py) collects all flagged vulnerabilities. It writes a structured summary to report.txt, documenting:

- The type of vulnerability found.
- The vulnerable URL and form action.
- The specific input field that accepted the payload.

3.2 Event Logs

The scanner provides real-time progress feedback in the console, indicating the current scanning phase:

```
[*] Starting scan on target: http://127.0.0.1:5000/
[*] Crawling site for links...
[+] Discovered pages: ['http://127.0.0.1:5000/', 'http://127.0.0.1:5000/search',
'http://127.0.0.1:5000/login']
[*] Parsing forms on discovered pages...
[*] Testing 'http://127.0.0.1:5000/search' for XSS...
[+] XSS Vulnerability Found on parameter: q
[*] Testing 'http://127.0.0.1:5000/login' for SQL Injection...
```

```
[+] SQL Injection Found on parameter: username
[*] Compilation complete. Writing results to report.txt...
[+] Report Generated.
```

3.3 Error Handling

To prevent crashes during scans, the application implements robust error handling:

- **Connection Errors:** If the target server is offline or unreachable, the scanner catches requests.exceptions.ConnectionError, prints an error message, and exits gracefully.
- **Timeout Errors:** Requests are configured with a timeout threshold (e.g., 5 seconds) to prevent the scanner from hanging indefinitely on slow endpoints.
- **Missing Form Elements:** If a form lacks an action or method attribute, the parser handles it gracefully by defaulting the action to the current page URL and the method to GET.

4. Performance

- **Network Overhead:** Because the scanner runs sequentially and targets discovered inputs specifically, it maintains a low network footprint, avoiding heavy traffic spikes.
- **Memory Footprint:** The application is highly lightweight, storing only a small list of URLs and form structures in memory.
- **Detection Accuracy:** The signature-based approach has 100% accuracy against simple, unescaped reflection (XSS) and verbose database error pages (SQLi). It does not produce false positives on the mock application.

5. Conclusion

The Web Vulnerability Scanner provides an automated, modular, and highly effective way to identify critical security vulnerabilities in web applications. By combining crawler, parser, and detection modules, it fulfills the core requirements of a basic automated security testing pipeline. The design ensures clear separation of concerns, allowing developer teams to easily maintain the code, add new security payloads, or integrate the tool into broader testing workflows.

6. References

- **OWASP Top 10:** Global standard for web application security risks (specifically A03:2021-Injection and A03:2021-Cross-Site Scripting).
- **Python Requests Documentation:** https://requests.readthedocs.io
- **BeautifulSoup 4 Documentation:** https://www.crummy.com/software/BeautifulSoup/bs4/doc/